



信息安全科技创新

2026年7月



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



基于内核模块的包过滤防火墙

陈秀真，姚立红

2026年7月



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

目录 Contents

- 1 计算机网络相关概念
- 2 网络防火墙功能
- 3 Linux的netfilter机制
- 4 内核模块包过滤防火墙原型
- 5 关键技术
- 6 内核模块包过滤防火墙原型的扩展开发



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



1. 计算机网络相关概念



怎么找到对方？

IP地址：202.120.2.119,……

域名（服务器）：www.sjtu.edu.cn

与设备上哪个应用通信？

端口号：http-80；https-443

怎么交互？

- QQ、微信聊天
- 网页浏览
- 电子邮件
- ……

→ 应用层协议

专用协议

通用协议：HTTP，
HTTPS，SMTP……



网络分层体系结构



- ⑤ **应用层(application layer)** : 提供应用程序便捷的网络服务调用
- ④ **传输层(transport layer)** : 将数据从源端口发送到目的端口 (进程到进程)
- ③ **网络层(network layer)** : 将数据包跨越网络从源设备发送到目的设备 (host to host)
- ② **数据链路层(data link layer)** : 实现相邻 (Neighboring) 网络实体间的数据传输
- ① **物理层(physical layer)** : 如何在连接各种计算机的传输媒体上传输数据比特流



示例：访问交大主页产生的网络报文 (wireshark)

无线局域网适配器 WLAN:

```

连接特定的 DNS 后缀 . . . . . :
本地链接 IPv6 地址 . . . . . : fe80::9d01:1e6f:c01c:4544%10
IPv4 地址 . . . . . : 192.168.1.21
子网掩码 . . . . . : 255.255.255.0
默认网关 . . . . . : 192.168.1.1
  
```

o.	Time	Source	Destination	Protocol	Length	Info
65	5.498536	192.168.1.21	202.120.2.119	TCP	66	56423 → 843 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_
66	5.582776	202.120.2.119	192.168.1.21	TCP	66	443 → 56423 [SYN, ACK] Seq=0 Ack=1 Win=8190 Len=0 MSS=1460 WS
67	5.583833	192.168.1.21	202.120.2.119	TCP	54	56423 → 443 [ACK] Seq=1 Ack=1 Win=131328 Len=0

< Frame 65: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{392EAD92-FF25-44B2-A11C-02169DC548D5}, Id 0

Ethernet II, Src: IntelCor_b3:d8:f8 (5c:5f:67:b3:d8:f8), Dst: 04:f9:f8:fd:3f:d3 (04:f9:f8:fd:3f:d3)

Destination: 04:f9:f8:fd:3f:d3 (04:f9:f8:fd:3f:d3)

Source: IntelCor_b3:d8:f8 (5c:5f:67:b3:d8:f8)

Type: IPv4 (0x0800)

Internet Protocol Version 4, Src: 192.168.1.21, Dst: 202.120.2.119

0100 = Version: 4

.... 0101 = Header Length: 20 bytes (5)

Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

Total Length: 52

Identification: 0x0fed (4065)

Flags: 0x40, Don't fragment

...0 0000 0000 0000 = Fragment Offset: 0

Time to Live: 64

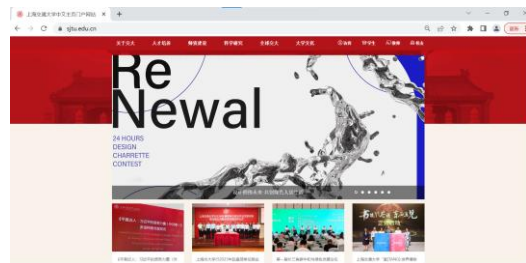
Protocol: TCP (6)

Header Checksum: 0x9c33 [validation disabled]

[Header checksum status: Unverified]

Source Address: 192.168.1.21

Destination Address: 202.120.2.119



```

Transmission Control Protocol, Src Port: 56423, Dst Port: 443, Seq: 8, Len: 0
Source Port: 56423
Destination Port: 443
[Stream index: 16]
[Conversation completeness: Incomplete, data (13)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 37378760
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment Number (raw): 0
1000 .... = Header Length: 32 bytes (8)
Flags: 0x0000 (NONE)
Window: 65535
[Calculated window size: 65535]
Checksum: 0x0000 [Unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options (12 bytes), Maximum segment size, No-operation (NOP), Window scale, No-operation (NOP), No-operation (NOP), SACK permitted
[Timestamp]
  
```

网络层：IP报文——源IP地址，目的IP地址，协议，源端口，目的端口

2. 网络防火墙功能



④ 基本概念

- **抽象**：防止网络攻击
- **功能**：按一定的策略管理网络访问，即进行**网络访问控制**、**审计**及**告警**等。

④ 逻辑结构

- **访问决策**：依据**访问控制规则**，做出当前网络访问是正当的、应该放行的，或不正当的、需阻断的判决。
- **访问实施**：对行程的访问控制判决，对一特定的网络访问进行控制，即放行或阻断该网络访问。

包过滤防火墙

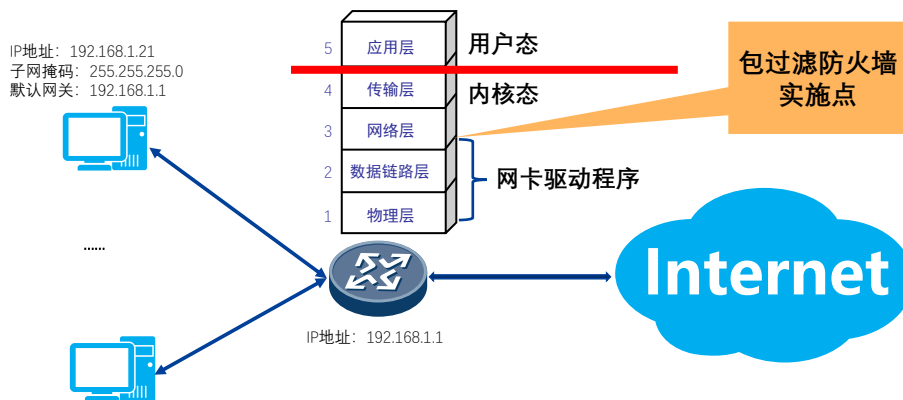


- ④ **包过滤防火墙的安放点**：通常嵌入在连接内外网的**路由器（或网关）**上实现。

④ 包过滤防火墙的工作原理：

- 包过滤操作是在**网络层**实现。
- 根据数据包的**源IP地址**、**目标IP地址**、**协议类型（TCP、UDP、ICMP等）**、**源端口**、**目的端口**、**ICMP消息类型**等报头信息及数据包传输方向等信息来，判断是否允许数据包通过。

包过滤防火墙的实施点



包过滤防火墙的特点

- ④ **接入方便:** 包过滤防火墙工作在**网络层**，与应用层不相关，用户不需要改变客户端的任何应用程序或设置，也无需对内部网络用户进行相关使用培训，因而很容易接入到现存的网络环境中。
- ④ **速度快:** 包过滤防火墙工作在网络层，至多分析所对应的传输层协议（即获得端口等相关的属性信息），协议处理比较简单，所以处理包的速度比应用代理防火墙快。
- ④ **实现简单:** 包过滤防火墙实现相对简单，甚至可以集成到原有的路由器中。目前很多网络路由器都有IP包过滤功能，它们在逻辑上可以认为是包过滤防火墙。

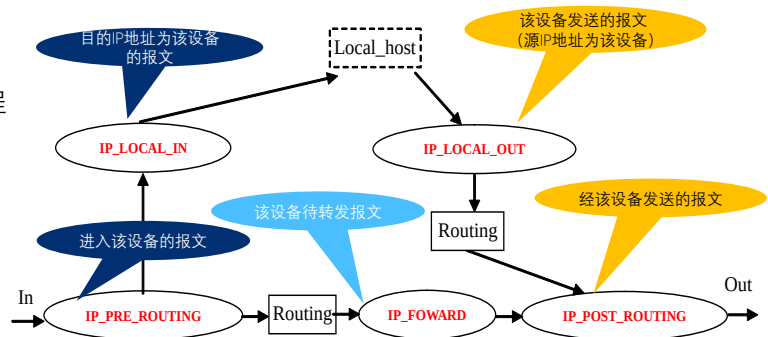
3. Linux的Netfilter机制

Netfilter的基本概念

- Netfilter机制的核心是一个**开放式的IP数据包处理框架**，该框架对外提供了操纵和处理IP数据的统一接口，编程人员可以利用该接口实现对IP包的控制以及其它新的处理方式。
- Linux系统自身借助该机制实现一些常见的IP包处理方式（**iptables**），包括重新实现了其内核包过滤防火墙（称之为**Linux内置防火墙**），及其它相关的处理功能，如网络地址转换**NAT**等。
- 第三方的软件开发者可以基于Netfilter提供的**IP报文处理接口**，开发相应的网络工具，包括网络防火墙、NAT、网络审计等。

Netfilter的核心思想

- 在网络IP协议层IP数据包的处理流程中，总结出几个**关键点（即钩子点）**，这些关键点提供了多种可能的IP数据包处理方式和开放接口。
- 安全管理员不但可以**配置**Netfilter以不同的方式处理IP数据包，也可利用Netfilter所提供的开放接口在IP数据包的协议处理流程中**实现**新的IP包处理方式。



IP数据包的处理流程中的五个钩子点

🔗 Netfilter的5个挂装点

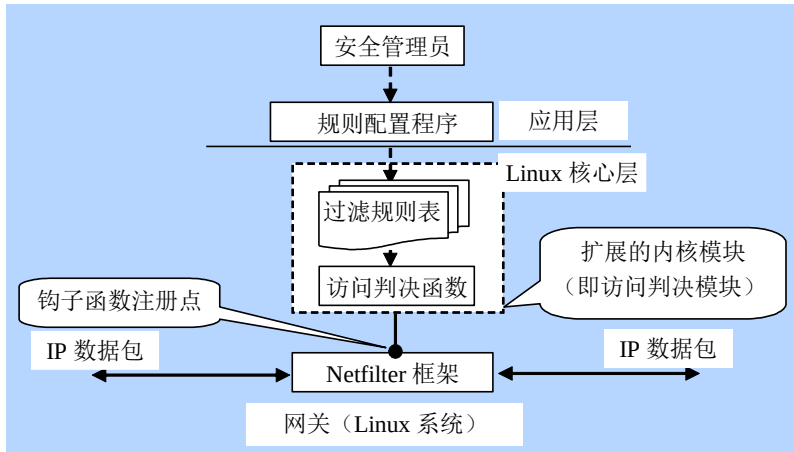
- **NF_INET_PRE_ROUTING**: 刚进入网络层的IP报文通过该点（即刚完成版本号、校验和等检测），**目的地址转换（DNAT、REDIRECT）**在此点进行；
- **NF_INET_LOCAL_IN**: 经路由查找后，送往本机的IP报文通过该点，**INPUT包过滤**在此进行；
- **NF_INET_FORWARD**: 经路由查找后，需经本机转发的IP报文通过该点，**FORWARD包过滤**在此点进行；
- **NF_INET_LOCAL_OUT**: 本机进程发出的IP报文通过该点，**OUTPUT包过滤**在此点进行；
- **NF_INET_POST_ROUTING**: 所有即将由本机网卡发送的IP报文通过该点，**源地址转换功能（SNAT、MASQUERADE）**在此点进行。

🔗 开放处理方式

- 可在Netfilter机制基础上自己开发**新的报文处理方式**，Netfilter将IP报文的**处理权**交给新开发的报文处理方式。
- 开放处理方式包括两种具体方式：
 - **钩子函数方式**: Netfilter机制为每种网络协议(IPv4、IPv6等)定义一套钩子，在数据报流过协议栈的某钩子点，挂接在该钩子上的钩子函数将会被调用。
 - **队列输出方式**: Netfilter提供队列输出功能，在这些关键点将所经过的IP数据包通过一定的方式直接交给应用层，程序员可以在应用层开发应用软件对这些数据包进行完全自主的处理，如丢弃、修改等。

4. 内核模块包过滤防火墙原型

运行原理



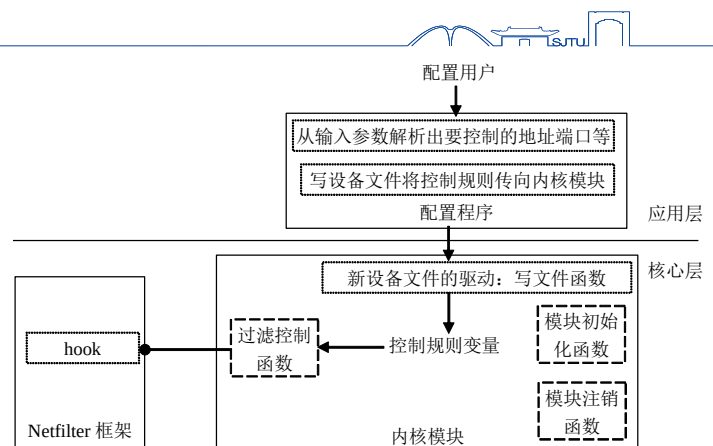
控制规则的配置:

- **白名单**
设置规则，如符合规则，则放行
- **黑名单**
设置规则，如符合规则，则丢弃

实现结构

原型系统分为**配置程序**、**内核模块**两个部分独立实现

- **配置程序**: 运行在**应用层**，用来设置包过滤规则，即设置报文相应的控制要素（参数），包括要控制的IP地址和端口等。（**黑名单**）

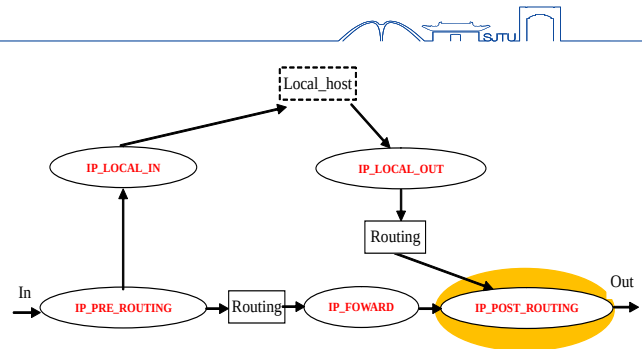


控制要素: **protocol**, **src_ip**, **src_port**, **dst_ip**, **dst_port**

通过虚拟设备的写操作"`write(fd, controlinfo, controlinfo_len);`", 将配置的**单条规则**发送至内核

- Linux内核模块：运行在**内核层**，完成包过滤防火墙的功能，该模块借助**注册Netfilter 钩子函数**的方式来实现对数据包的过滤和控制。

```
myhook.hook=hook_func;
myhook.hooknum=NF_INET_POST_ROUTING;
myhook.pf=PF_INET;
myhook.priority=NF_IP_PRI_FIRST;
```



运行方式

- 插入内核模块：insmod
- 通过命令行参数，输入要被**丢弃**的数据包的协议类型、源、目的IP地址以及源、目的端口号

`./configure -p protocol -x src_ip -y dst_ip -m src_port -n dst_port`

各选项的具体含义如下：

- `-p protocol` 指明要控制的协议（或网络应用）类型，具体为 tcp、udp、ping（**icmp 0/8**）三种之一；
- `-x src_ip` 指明要控制报文的源IP地址；
- `-y dst_ip` 指明要控制报文的目标IP地址；
- `-m src_port` 指明要控制报文的源端口；
- `-n dst_port` 指明要控制报文的目标端口；

——若某个选项省略则取其默认值为0，0表示对任意值相匹配，即表示控制的报文覆盖该选项的所有值域。

5.关键技术

基于内核模块的Netfilter链钩子函数挂装/卸载

```
struct nf_hook_ops
{
    struct list_head list;           //链表成员
    /* User fills in from here down. */
    nf_hookfn *hook;                //钩子函数指针
    struct module *owner;
    int pf;                          //协议族, 对于ipv4而言, 是PF_INET
    int hooknum;                    //hook类型
    /* Hooks are ordered in ascending priority. */
    int priority;                   //优先级
};
```

```
/* Supported address families. */
#define AF_UNSPEC 0
#define AF_UNIX 1 /* Unix domain sockets */
#define AF_LOCAL 1 /* POSIX name for AF_UNIX */
#define AF_INET 2 /* Internet IP Protocol */
#define AF_AX25 3 /* Amateur Radio AX.25 */
#define AF_IPX 4 /* Novell IPX */
#define AF_APPLETALK 5 /* AppleTalk DDP */
#define AF_NETROM 6 /* Amateur Radio NET/ROM */
#define AF_BRIDGE 7 /* Multiprotocol bridge */
#define AF_ATMPVC 8 /* ATM PVCs */
#define AF_X25 9 /* Reserved for X.25 project */
#define AF_INET6 10 /* IP version 6 */
#define AF_ROUTE 11 /* Amateur Radio X.25 PLP */
#define AF_DECnet 12 /* Reserved for DECnet project */
```

```
enum nf_inet_hooks {
    NF_INET_PRE_ROUTING,
    NF_INET_LOCAL_IN,
    NF_INET_FORWARD,
    NF_INET_LOCAL_OUT,
    NF_INET_POST_ROUTING,
    NF_INET_NUMHOOKS
};
```

```
enum nf_ip_hook_priorities {
    NF_IP_PRI_FIRST = INT_MIN,
    NF_IP_PRI_CONTRACK_DEFRAG = -400,
    NF_IP_PRI_RAW = -300,
    NF_IP_PRI_SELINUX_FIRST = -225,
    NF_IP_PRI_CONTRACK = -200,
    NF_IP_PRI_MANGLE = -150,
    NF_IP_PRI_NAT_DST = -100,
    NF_IP_PRI_FILTER = 0,
    NF_IP_PRI_SECURITY = 50,
    NF_IP_PRI_NAT_SRC = 100,
    NF_IP_PRI_SELINUX_LAST = 225,
    NF_IP_PRI_CONTRACK_CONFIRM = INT_MAX,
    NF_IP_PRI_LAST = INT_MAX, };
```

在内核模块初始化函数module_init()中:

```
struct nf_hook_ops myhook;

myhook.hook=hook_func;
myhook.hooknum=NF_INET_POST_ROUTING;
myhook.pf=PF_INET;
myhook.priority=NF_IP_PRI_FIRST;

nf_register_net_hook(&init_net,&myhook);
```

在内核模块卸载函数module_exit()中:

```
nf_unregister_net_hook(&init_net,&myhook);
```

基于虚拟设备的应用层-内核通信

• 应用层

• 创建虚拟设备:

- 设备类型: c-字符设备, b-块设备
- 主设备号: 用于区别设备类型
- 次设备号: 用于标识同类型中的不同设备

```
if (stat("/dev/controlinfo",&buf) != 0){
    if (system("mknod /dev/controlinfo c 124 0") == -1){
        printf("Can't create the device file ! \n");
        printf("Please check and try again! \n");
        exit(1);
    }
}
```

• 打开设备:

```
fd = open("/dev/controlinfo", O_RDWR, S_IRUSR | S_IWUSR);
```

• 调用write函数, 将待传递数据写入虚拟设备

```
write(fd, controlinfo, controlinfo_len);
```

• 内核层

- 内核模块初始化时，注册对应设备的操作函数

```
struct file_operations fops = {
    .owner=THIS_MODULE,
    .write=write_controlinfo,
};
```

- 操作函数（读、写等）实现：

- 例 write_controlinfo()：接收来自于应用层传递的数据，解析成一条控制规则至数据结构中
- 注意：使用“`unsigned long copy_from_user(void * to, const void __user * from, unsigned long n)`”，将from指针指向的用户空间地址开始的连续n个字节数据传送到to指针指向的内核空间地址

- 在注销内核模块时，同时注销设备操作函数：

```
unregister_chrdev(124, "/dev/controlinfo")
```

注意：应用层、内核的虚拟设备名称、主设备号保持一致

6. 内核包过滤防火墙扩展开发

🕒 背景：原型系统的控制功能简单

- **控制规则简单**，只是依据通信双方的IP地址和端口进行检查和控制，实际上包过滤防火墙还可以依据其它要素进行控制。比如，基于时间段进行控制，比如在休息日时间不准从外网访问内网等。
- 以几个简单变量存储相应的控制信息，相当于**只能支持一条包过滤规则**。一个实用的包过滤防火墙需要同时支持多条包过滤规则。



● 开发目标：

- **检查和控制要素的扩展。**除实现基于IP地址、端口的检查和控制外，还能使基于时间段、网络接口、ICMP报文的子类型等进行报文的检查和控制。
- **多包过滤规则的扩展。**以表的形式存储包过滤规则，能够支持用户配置多条包过滤规则，使内核模块防火墙能够同时按多条包过滤规则进行报文检查和过滤控制。
- **友好的包过滤规则的配置和管理界面。**支持包过滤的规则导入、导出，添加、编辑、删除等基本功能。

谢谢！

